

2026-05-06-p1-f15-subagent-team

P1-F15 — Subagent Team Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development. Steps use checkbox (- []) syntax for tracking.

Goal: Ship a real, end-to-end subagent dispatch system. The agent calls a new `task` tool that fans out work to one or more subagents, each running its own (inner) agent loop with a real `llm.Provider` invocation and a real tool registry. Default execution is **in-process goroutines**; when `isolation: "worktree"` is requested, the subagent runs as a **subprocess** (re-exec of the host binary with `HELIX_SUBAGENT_INVOCATION=1`, mirroring F14's helper-mode dispatch) inside an F04-managed worktree. Results stream back over a `<-chan SubagentResult` and the channel closes when the last subagent completes; a `WaitAll` helper drains synchronously. A `/subagents` slash command (`list/status <id>/kill <id>`) provides inspection and termination. **No cobra subcommand** (Q5=B).

Architecture: New `internal/agent/subagent/` package with `types.go` (incl. `FakeLLMProvider TEST PROVIDER`), `inprocess_spawner.go`, `subprocess_spawner.go`, `manager.go`, `worktree_integration.go`, `helper_mode.go` (+ tests). New tool at `internal/tools/task_tool.go` (NOT inside the subagent subpackage — keeps the registry self-contained, mirrors F13's `LSPTool` placement). New slash at `internal/commands/subagents_command.go`. `internal/tools/registry.go` gets a single new method `SetSubagentManager(*subagent.SubagentManager)` (mirrors F13's `SetLSPManager` and F14's `SetSandboxManager`) which lazily registers the task tool. `cmd/cli/main.go` adds three lines: a `subagent.IsSubagentInvocation()` early-main check (FIRST statement, before `sandbox.IsHelperInvocation()`), a manager construction, and the slash registration.

Tech Stack: Go 1.26, testify v1.11, spf13/cobra v1.8 — all already present. **NO new external deps:** subagent dispatch uses `os/exec + os.Executable()` from the standard library; helper-mode uses `os.Getenv + os.Stdin/Stdout` (stdlib); JSON encoding/decoding via `encoding/json` (stdlib); UUID generation reuses `github.com/google/uuid` already in `go.mod` for F03/F07/F11. No `go get` needed. Confirmed: `go.mod` already carries `github.com/google/uuid v1.6.0`, `github.com/stretchr/testify v1.11.1`, and `go.uber.org/zap v1.27.0`.

Spec: `docs/superpowers/specs/2026-05-06-p1-f15-subagent-team-design.md` (commit `cb078c6`)

Working directory for go commands: `helix_code/`. Git from meta-repo root.

Anti-bluff smoke (FULL 4-term applied to F15 surface):

```
cd HelixCode && grep -rn "simulated\\|for now\\|TODO implement\\|
placeholder" \
internal/agent/subagent internal/tools/task_tool.go internal/
commands/subagents_command.go \
&& echo BLUFF || echo clean
```

Must always print clean.

Anti-bluff hot zone: §5.2 of the spec — a “subagent” can degenerate into a no-op in three ways: (a) hardcoded “I would have done X” string, (b) goroutine that echoes the prompt unchanged, (c) struct that “represents” a subagent but never calls the LLM. The four real-execution criteria (in-process LLM call count ≥ 1 , subprocess fork-exec with distinct PID, worktree dir actually exists at a different path from parent cwd, channel-streamed-not-batch-collected ordering) are each tested with both unit assertions AND a Challenge phase. The FakeLLMProvider is the ONE TEST-ONLY type that lives in `types.go` (not `_test.go`) so the integration binary and Challenge harness can both link it; its anti-misuse comment is self-tested via `grep`, and the bluff scanner gets a special-case allowlist for it.

Why this is consequential: subagent dispatch is the most architecturally invasive feature in Phase 1 — it adds a second kind of process (the subprocess subagent), a streaming concurrency model (the result channel), and a worktree-isolation seam (F04 integration). Any of those silently degrading to a no-op produces a feature that “works” in tests but is useless in practice. The Phase A in-process FakeLLM call-count assertion is the single most important test in F15.

Task list



P1-F15-T01 — bootstrap evidence + advance PROGRESS to F15



P1-F15-T02 — `internal/agent/subagent/types.go`: SubagentTask + SubagentResult + Isolation/SubagentState enums + SubagentSpawner interface + error sentinels + FakeLLMProvider TEST PROVIDER (TDD)



P1-F15-T03 — `internal/agent/subagent/inprocess_spawner.go`: goroutine-based spawner; inner agent loop invokes `llm.Provider.Generate` (TDD; FakeLLM call-count assertion)



P1-F15-T04 — `internal/agent/subagent/subprocess_spawner.go`: helper-mode re-exec spawner; sentinel env-var dispatch; JSON stdout decode (TDD; `argv+env` assertions on injected Executor)



P1-F15-T05 — `internal/agent/subagent/manager.go`: SubagentManager with dispatch + streaming aggregation + max-concurrency + kill-by-id (TDD; channel-close-on-error invariant)



P1-F15-T06 — `internal/agent/subagent/worktree_integration.go`: F04 worktree integration for `isolation=worktree` (TDD; `real git init tempdir`)



P1-F15-T07 — `internal/tools/task_tool.go`: TaskTool implementing `tools.Tool` interface, registered name `task` (TDD)



P1-F15-T08 — `internal/agent/subagent/helper_mode.go` + `main.go`
integration: `IsSubagentInvocation` + `RunAsSubagent` + early-main dispatch



P1-F15-T09 — `internal/commands/subagents_command.go`: `/subagents` slash
(`list` / `status` / `kill`) (TDD; CONST-042 anti-leak: `status` shows description not prompt body)



P1-F15-T10 — `main.go` wiring (`Manager` + `tool` + `slash` + `helper-mode`) + integration tests
(gated, SKIP-OK on missing git)



P1-F15-T11 — Challenge harness (Phase A in-process always; Phase B subprocess always;
Phase C worktree gated on git; Phase D real-LLM gated on `ANTHROPIC_API_KEY`)



P1-F15-T12 — Feature 15 close-out + push 4 remotes non-force

Task 1: Bootstrap

Append F15 evidence section header (spec `cb078c6`), update PROGRESS current focus to F15,
insert F15 task list (12 items) after F14's. Commit docs (P1-F15-T01): `bootstrap`
Phase 1 / Feature 15 evidence + advance PROGRESS.

Task 2: `types.go` (TDD)

Files: new `helix_code/internal/agent/subagent/types.go`, new
`helix_code/internal/agent/subagent/types_test.go`.

Define `Isolation` enum (`IsolationNone`, `IsolationWorktree`), `SubagentState` enum (`StatePending`, `StateRunning`, `StateSucceeded`, `StateFailed`, `StateKilled`, `StateTimedOut`), `SubagentTask` struct, `SubagentResult` struct, `SubagentSpawner` interface, error sentinels (`ErrMaxConcurrency`, `ErrSubagentNotFound`, `ErrWorktreeUnavailable`, `ErrInvalidIsolation`, `ErrSubagentTimeout`, `ErrSubagentKilled`), and `FakeLLMProvider` (TEST PROVIDER per spec §5.2). Add `DefaultSubagentTask()` (sets `Isolation`: `IsolationNone`, `Timeout`: `5*time.Minute`).

Failing tests FIRST:

```
func TestIsolation_DefaultIsNone(t *testing.T) {
    require.Equal(t, IsolationNone,
        DefaultSubagentTask().Isolation)
}

func TestSubagentTask_TimeoutDefaultsTo5Min(t *testing.T) {
    require.Equal(t, 5*time.Minute,
        DefaultSubagentTask().Timeout)
}

func TestSubagentTask_TimeoutClampedTo30Min(t *testing.T) {
```

```

require.LessOrEqual(t, ClampTimeout(60*time.Minute),
    30*time.Minute)
require.Equal(t, 1*time.Second,
    ClampTimeout(1*time.Second)) // no lower clamp
}

func TestFakeLLMProvider_RecordsCallCount(t *testing.T) {
    f := NewFakeLLMProvider(map[string]string{"hi": "hello"})
    _, err := f.Generate(context.Background(),
        &llm.LLMRequest{Prompt: "hi"})
    require.NoError(t, err)
    _, _ = f.Generate(context.Background(),
        &llm.LLMRequest{Prompt: "hi"})
    require.Equal(t, 2, f.GenerateCallCount())
}

func TestFakeLLMProvider_LastPrompt(t *testing.T) {
    f := NewFakeLLMProvider(nil)
    _, _ = f.Generate(context.Background(),
        &llm.LLMRequest{Prompt: "second"})
    require.Equal(t, "second", f.LastPrompt())
}

func TestFakeLLMProvider_FallbackEcho(t *testing.T) {
    f := NewFakeLLMProvider(nil)
    resp, _ := f.Generate(context.Background(),
        &llm.LLMRequest{Prompt: "x"})
    require.Equal(t, "FAKE-LLM-ECHO: x", resp.Text)
}

func TestFakeLLMProvider_HasAntiMisuseComment(t *testing.T) {
    src, err := os.ReadFile("types.go")
    require.NoError(t, err)
    require.Contains(t, string(src), "TEST-ONLY llm.Provider")
    require.Contains(t, string(src),
        "MUST NOT be referenced from production code")
}

```

Subject: feat(P1-F15-T02): subagent types + Isolation/State enums + FakeLLMProvider TEST PROVIDER.

Task 3: inprocess_spawner.go (TDD)

Files: new helix_code/internal/agent/subagent/inprocess_spawner.go, new helix_code/internal/agent/subagent/inprocess_spawner_test.go.

InProcessSpawner runs the subagent's inner agent loop in a goroutine. The inner loop is a thin wrapper that: 1. Calls provider.Generate(ctx, &llm.LLMRequest{Prompt:

task.Prompt)). 2. Counts the response as the subagent's Output. 3. Records start/finish timestamps. 4. (v1) Does NOT iterate tool calls — a single LLM call per subagent. Multi-turn loops are F15.5; v1 keeps the inner loop simple to make the call-count assertion unambiguous.

```
type InProcessSpawner struct {
    Provider      llm.Provider
    ToolRegistry  *tools.ToolRegistry
    Logger        *zap.Logger
}

func NewInProcessSpawner(p llm.Provider, r *tools.ToolRegistry,
    log *zap.Logger) *InProcessSpawner
func (s *InProcessSpawner) Kind() string { return "in-process" }
func (s *InProcessSpawner) Spawn(ctx context.Context, sub
    *Subagent, sink chan<- SubagentResult) error
```

Tests:

```
func TestInProcessSpawner_InvokesLLMProvider(t *testing.T) {
    fake := NewFakeLLMProvider(map[string]string{"do thing":
        "did thing"})
    s := NewInProcessSpawner(fake, nil, zap.NewNop())
    sub := &Subagent{ID: "x", Task: &SubagentTask{Prompt: "do
        thing"}}
    sink := make(chan SubagentResult, 1)
    require.NoError(t, s.Spawn(context.Background(), sub, sink))
    res := <-sink
    require.Equal(t, "did thing", res.Output,
        "anti-bluff: output MUST be LLM response, not prompt
        echo")
    require.Equal(t, 1, fake.GenerateCallCount(), "anti-bluff:
        provider MUST be called exactly once")
}

func TestInProcessSpawner_OutputIsLLMResponse_NotPromptEcho(t
    *testing.T) {
    fake := NewFakeLLMProvider(nil) // fallback echo: "FAKE-LLM-
        ECHO: <prompt>"
    s := NewInProcessSpawner(fake, nil, zap.NewNop())
    sub := &Subagent{ID: "x", Task: &SubagentTask{Prompt: "abc"}}
    sink := make(chan SubagentResult, 1)
    _ = s.Spawn(context.Background(), sub, sink)
    res := <-sink
    require.Equal(t, "FAKE-LLM-ECHO: abc", res.Output)
    require.NotEqual(t, "abc", res.Output, "result.Output MUST
        NOT be prompt echo (bluff vector b)")
}

func TestInProcessSpawner_PropagatesContextCancel(t *testing.T) {
```

```

fake := &slowFakeLLMProvider{delay: 5 * time.Second}
s := NewInProcessSpawner(fake, nil, zap.NewNop())
ctx, cancel := context.WithTimeout(context.Background(),
    50*time.Millisecond)
defer cancel()
sub := &Subagent{ID: "x", Task: &SubagentTask{Prompt:
    "slow"}}
sink := make(chan SubagentResult, 1)
_ = s.Spawn(ctx, sub, sink)
res := <-sink
require.Equal(t, StateTimedOut, res.State)
}

```

Subject: feat(P1-F15-T03): InProcessSpawner with real llm.Provider invocation + ctx cancel.

Task 4: subprocess_spawner.go (TDD)

Files: new helix_code/internal/agent/subagent/subprocess_spawner.go, new helix_code/internal/agent/subagent/subprocess_spawner_test.go.

SubprocessSpawner fork-execs os.Executable() with HELIX_SUBAGENT_INVOCATION=1 + HELIX_SUBAGENT_PAYLOAD=<base64-j son>. The injected Executor seam lets unit tests assert argv + env without spawning a real subprocess.

```

type Executor interface {
    Run(ctx context.Context, name string, args []string, env
        []string, cwd string,
        stdout, stderr *bytes.Buffer) (exitCode int, err error)
}

type SubprocessSpawner struct {
    HostBinary string // defaults to os.Executable()
    Executor   Executor // injectable; defaults to
        defaultExecExecutor
    Logger     *zap.Logger
}

func NewSubprocessSpawner(log *zap.Logger) (*SubprocessSpawner,
    error)
func (s *SubprocessSpawner) Kind() string { return "subprocess" }
func (s *SubprocessSpawner) Spawn(ctx context.Context, sub
    *Subagent, sink chan<- SubagentResult) error

```

Tests:

```

func TestSubprocessSpawner_BuildsExecCommandWithSentinelEnv(t
    *testing.T) {
    rec := &recordingExecutor{stdout: makeJSONResult("subagent-
        x", StateSucceeded, "ok")}
    s := &SubprocessSpawner{HostBinary: "/usr/bin/helixcode",
        Executor: rec, Logger: zap.NewNop()}
    sub := &Subagent{ID: "subagent-x", Task:
        &SubagentTask{Prompt: "p", WorktreePath: "/tmp/wt"}}
    sink := make(chan SubagentResult, 1)
    require.NoError(t, s.Spawn(context.Background(), sub, sink))
    require.Equal(t, "/usr/bin/helixcode", rec.lastName)
    require.Contains(t, rec.lastEnv,
        "HELIX_SUBAGENT_INVOCATION=1")
    require.True(t, hasEnvKey(rec.lastEnv,
        "HELIX_SUBAGENT_PAYLOAD"))
}

```

```

func TestSubprocessSpawner_SetsCwdToWorktreePath(t *testing.T) {
    rec := &recordingExecutor{stdout: makeJSONResult("x",
        StateSucceeded, "ok")}
    s := &SubprocessSpawner{HostBinary: "/bin/x", Executor: rec,
        Logger: zap.NewNop()}
    sub := &Subagent{ID: "x", Task: &SubagentTask{},
        WorktreePath: "/tmp/wt-x"}
    _ = s.Spawn(context.Background(), sub, make(chan
        SubagentResult, 1))
    require.Equal(t, "/tmp/wt-x", rec.lastCwd)
}

```

```

func TestSubprocessSpawner_DecodesStdoutJSON(t *testing.T) {
    rec := &recordingExecutor{stdout: makeJSONResult("y",
        StateSucceeded, "the-answer")}
    s := &SubprocessSpawner{HostBinary: "/bin/x", Executor: rec,
        Logger: zap.NewNop()}
    sub := &Subagent{ID: "y", Task: &SubagentTask{}}
    sink := make(chan SubagentResult, 1)
    _ = s.Spawn(context.Background(), sub, sink)
    res := <-sink
    require.Equal(t, "the-answer", res.Output)
    require.Equal(t, StateSucceeded, res.State)
}

```

```

func TestSubprocessSpawner_HandlesMalformedStdout(t *testing.T) {
    rec := &recordingExecutor{stdout: []byte("not-json")}
    s := &SubprocessSpawner{HostBinary: "/bin/x", Executor: rec,
        Logger: zap.NewNop()}
    sub := &Subagent{ID: "z", Task: &SubagentTask{}}
    sink := make(chan SubagentResult, 1)
}

```

```

    _ = s.Spawn(context.Background(), sub, sink)
    res := <-sink
    require.Equal(t, StateFailed, res.State)
    require.Contains(t, res.Error, "subagent stdout malformed")
}

```

Subject: feat(P1-F15-T04): SubprocessSpawner with sentinel env var + JSON stdout decode.

Task 5: manager.go (TDD)

Files: new helix_code/internal/agent/subagent/manager.go, new helix_code/internal/agent/subagent/manager_test.go.

SubagentManager.Dispatch (per spec §4.2): acquire semaphore → assign UUID → set timeout-bound ctx → if isolation=worktree, call wtMgr.EnterWorktree, pick subprocess spawner, else pick in-process → register sub in m.running → spawn goroutine that calls spawner.Spawn and defer close(ch) + defer release sem.

Tests:

```

func TestManager_Dispatch_ReturnsIDAndChannel(t *testing.T) {
    fake := NewFakeLLMProvider(map[string]string{"hi": "hello"})
    m, _ :=
        NewSubagentManager(SubagentManagerOptions{LLMProvider:
            fake, MaxConcurrency: 5})
    id, ch, err := m.Dispatch(context.Background(),
        &SubagentTask{Prompt: "hi"})
    require.NoError(t, err)
    require.NotEmpty(t, id)
    res := <-ch
    require.Equal(t, "hello", res.Output)
    _, ok := <-ch
    require.False(t, ok, "channel MUST close after final result")
}

```

```

func TestManager_Dispatch_RespectsMaxConcurrency(t *testing.T) {
    fake := &slowFakeLLMProvider{delay: 1 * time.Second}
    m, _ :=
        NewSubagentManager(SubagentManagerOptions{LLMProvider:
            fake, MaxConcurrency: 2})
    _, _, _ = m.Dispatch(context.Background(),
        &SubagentTask{Prompt: "a"})
    _, _, _ = m.Dispatch(context.Background(),
        &SubagentTask{Prompt: "b"})
    ctx, cancel := context.WithTimeout(context.Background(),
        100*time.Millisecond)
    defer cancel()
    _, _, err := m.Dispatch(ctx, &SubagentTask{Prompt: "c"})
}

```

```

    require.ErrorIs(t, err, ErrMaxConcurrency)
}

func TestManager_Dispatch_ChannelClosesEvenOnSpawnError(t
    *testing.T) {
    m, _ :=
        NewSubagentManager(SubagentManagerOptions{LLMProvider:
            NewFakeLLMProvider(nil), InProcess: &erroringSpawner{}})
    _, ch, err := m.Dispatch(context.Background(),
        &SubagentTask{Prompt: "x"})
    require.NoError(t, err)
    res := <-ch
    require.Equal(t, StateFailed, res.State)
    _, ok := <-ch
    require.False(t, ok, "channel MUST close even when spawner
        errors")
}

func TestManager_Dispatch_StreamsResultsAsTheyComplete(t
    *testing.T) {
    // Anti-bluff (4): two subagents, A fast B slow; A's result
    // lands first.
    fast := NewFakeLLMProvider(map[string]string{"A": "ra"});
    fast.SetDelay("A", 10*time.Millisecond)
    slow := NewFakeLLMProvider(map[string]string{"B": "rb"});
    slow.SetDelay("B", 200*time.Millisecond)
    // ... assert ordering on a merged channel
}

func TestManager_Kill_CancelsRunningSubagent(t *testing.T) { /*
    cancel ctx; result.State=StateKilled within 500ms */ }
func TestManager_Kill_UnknownID_ReturnsErrSubagentNotFound(t
    *testing.T) {
    m, _ :=
        NewSubagentManager(SubagentManagerOptions{LLMProvider:
            NewFakeLLMProvider(nil)})
    require.ErrorIs(t, m.Kill("nonsense"), ErrSubagentNotFound)
}

func TestManager_WaitAll_DrainsChannelToClose(t *testing.T) { /*
    WaitAll returns all N results in order */ }
func TestManager_Close_DrainsRunningSubagents(t *testing.T) { /*
    in-flight subagents finish or get cancelled */ }

```

Subject: feat(P1-F15-T05): SubagentManager with streaming dispatch + max-concurrency + kill-by-id.

Task 6: `worktree_integration.go` (TDD)

Files: `new helix_code/internal/agent/subagent/worktree_integration.go`, `new helix_code/internal/agent/subagent/worktree_integration_test.go`.

Wraps F04's `worktree.Manager.EnterWorktree`. The function `prepareWorktreeForSubagent(ctx, wtMgr, subID, baseBranch)` returns `(path string, err error)`. Errors are wrapped with `ErrWorktreeUnavailable` when `wtMgr` is `nil`.

Test (uses `real git init tempdir`):

```
func TestWorktreeIntegration_CreatesRealWorktree(t *testing.T) {
    if _, err := exec.LookPath("git"); err != nil {
        t.Skip("SKIP-OK: P1-F15 git not on PATH (install: apt
            install git OR dnf install git)")
    }
    repo := t.TempDir()
    runGit(t, repo, "init")
    runGit(t, repo, "commit", "--allow-empty", "-m", "init")
    wtMgr := worktree.NewManager(repo)

    path, err :=
        prepareWorktreeForSubagent(context.Background(), wtMgr,
            "sub-test-1", "")
    require.NoError(t, err)
    require.DirExists(t, path)
    require.NotEqual(t, repo, path, "worktree path must differ
        from parent cwd")

    // Anti-bluff: git rev-parse --show-toplevel inside the
    // worktree returns the worktree itself.
    out, err := exec.Command("git", "-C", path, "rev-parse", "--
        show-toplevel").Output()
    require.NoError(t, err)
    require.Equal(t, path, strings.TrimSpace(string(out)))
}

func TestWorktreeIntegration_NilManager_ReturnsErrWorktreeUnavailable(t
    *testing.T) {
    _, err := prepareWorktreeForSubagent(context.Background(),
        nil, "x", "")
    require.ErrorIs(t, err, ErrWorktreeUnavailable)
}
```

Subject: feat(P1-F15-T06): F04 worktree integration for
isolation=worktree (real git tempdir test).

Task 7: task_tool.go (TDD)

Files: new helix_code/internal/tools/task_tool.go, new helix_code/internal/tools/task_tool_test.go.

Lives in package tools (NOT subagent) so it can directly implement the Tool interface without an adapter. Imports dev.helix.code/internal/agent/subagent. New CategorySubagent ToolCategory = "subagent" constant added to registry.go.

```
type TaskTool struct { manager *subagent.SubagentManager }

func NewTaskTool(m *subagent.SubagentManager) *TaskTool
func (t *TaskTool) Name() string { return "task" }
func (t *TaskTool) Description() string // "Dispatch a subagent
    to perform a task..."
func (t *TaskTool) Schema() ToolSchema
func (t *TaskTool) Category() ToolCategory { return
    CategorySubagent }
func (t *TaskTool) Validate(params map[string]interface{}) error
func (t *TaskTool) Execute(ctx context.Context, params
    map[string]interface{}) (interface{}, error)
```

Schema (Q4=B): - description (string, required) - prompt (string, required) - isolation (string enum "none"|"worktree", default "none") - subagent_type (string, optional) - timeout_seconds (int, default 300, max 1800)

Tests:

```
func TestTaskTool_Name_IsTask(t *testing.T) {
    require.Equal(t, "task", (&TaskTool{}).Name())
}

func TestTaskTool_IsolationDefaultsNone(t *testing.T) {
    rec := &recordingManager{}
    tool := &TaskTool{manager: rec}
    _, _ = tool.Execute(context.Background(),
        map[string]interface{}{
            "description": "d", "prompt": "p",
        })
    require.Equal(t, subagent.IsolationNone,
        rec.lastTask.Isolation)
}

func TestTaskTool_RejectsInvalidIsolation(t *testing.T) {
    err := (&TaskTool{}).Validate(map[string]interface{}{
        "description": "d", "prompt": "p", "isolation":
        "bogus"})
    require.ErrorIs(t, err, subagent.ErrInvalidIsolation)
}
```

```

func TestTaskTool_ResultMapShape(t *testing.T) {
    fake := subagent.NewFakeLLMProvider(map[string]string{"hi":
        "ok"})
    m, _ :=
        subagent.NewSubagentManager(subagent.SubagentManagerOptions{LLMPro
            fake, MaxConcurrency: 5})
    tool := NewTaskTool(m)
    out, err := tool.Execute(context.Background(),
        map[string]interface{}{
            "description": "test", "prompt": "hi",
        })
    require.NoError(t, err)
    res := out.(map[string]interface{})
    require.NotEmpty(t, res["id"])
    require.Equal(t, "succeeded", res["state"])
    require.Equal(t, "ok", res["output"])
    require.Equal(t, "none", res["isolation"])
    require.NotContains(t, res, "worktree_path") // only present
        for worktree isolation
}

```

Subject: feat(P1-F15-T07): TaskTool implementing tools.Tool as task (claude-code-compatible name).

Task 8: helper_mode.go + main.go integration (TDD)

Files: new helix_code/internal/agent/subagent/helper_mode.go, new helix_code/internal/agent/subagent/helper_mode_test.go, modify helix_code/cmd/cli/main.go.

```

const subagentInvocationEnvVar = "HELIX_SUBAGENT_INVOCATION"
const subagentPayloadEnvVar    = "HELIX_SUBAGENT_PAYLOAD"

func IsSubagentInvocation() bool {
    return os.Getenv(subagentInvocationEnvVar) == "1"
}

func RunAsSubagent() (exitCode int) {
    payload, err :=
        decodeSubagentPayload(os.Getenv(subagentPayloadEnvVar))
    if err != nil { fmt.Fprintln(os.Stderr, err); return 2 }
    provider, err := buildProviderFromEnv() // F12 env wiring
    if err != nil { /* fail-closed → encode error result */ }
    // build minimal tool registry (no `task` recursion in v1)
    // run inner agent loop
    // print SubagentResult JSON to stdout, return 0/1
}

```

main.go modification — three lines (per spec §4.1):

```
// FIRST statement of main():
if subagent.IsSubagentInvocation() {
    os.Exit(subagent.RunAsSubagent())
}
// existing line:
if sandbox.IsHelperInvocation() {
    os.Exit(sandbox.RunAsHelper())
}
// ... existing bootstrap ...
saMgr, err :=
    subagent.NewSubagentManager(subagent.SubagentManagerOptions{
        LLMPProvider: provider, ToolRegistry: toolReg,
        WorktreeManager: wtMgr,
        MaxConcurrency: 5, DefaultTimeout: 5 * time.Minute,
    })
if err == nil {
    toolReg.SetSubagentManager(saMgr)
    defer saMgr.Close(context.Background())
}
slashRegistry.Register(commands.NewSubagentsCommand(saMgr))
```

Tests:

```
func TestHelperMode_IsSubagentInvocation_ReadsEnvVar(t
    *testing.T) {
    t.Setenv("HELIX_SUBAGENT_INVOCATION", "1")
    require.True(t, IsSubagentInvocation())
    t.Setenv("HELIX_SUBAGENT_INVOCATION", "")
    require.False(t, IsSubagentInvocation())
}

func TestHelperMode_RunAsSubagent_DecodesPayloadFromEnv(t
    *testing.T) {
    payload := encodeSubagentPayload(&SubagentPayload{ID: "x",
        Prompt: "hi"})
    t.Setenv("HELIX_SUBAGENT_INVOCATION", "1")
    t.Setenv("HELIX_SUBAGENT_PAYLOAD", payload)
    // capture stdout, run RunAsSubagent, parse result, assert
    JSON shape
}
```

Subject: feat(P1-F15-T08): IsSubagentInvocation + RunAsSubagent
helper-mode + main.go early-dispatch.

Task 9: /subagents slash command (TDD)

Files: new helix_code/internal/commands/subagents_command.go, new helix_code/internal/commands/subagents_command_test.go.

Mirrors F14 /sandbox command pattern: defines a small SubagentManager interface in the commands package so the slash is testable with a fake.

```
type SubagentManager interface {
    List() []subagent.SubagentInfo
    Status(id string) (subagent.SubagentInfo, error)
    Kill(id string) error
}

type SubagentsCommand struct { manager SubagentManager }

func NewSubagentsCommand(m SubagentManager) *SubagentsCommand
func (c *SubagentsCommand) Name() string { return "subagents" }
func (c *SubagentsCommand) Aliases() []string { return nil }
func (c *SubagentsCommand) Description() string { return
    "Inspect or kill running subagents." }
func (c *SubagentsCommand) Usage() string { return "/subagents
    [list|status <id>|kill <id>]" }
func (c *SubagentsCommand) Execute(ctx context.Context, cc
    *CommandContext) (*CommandResult, error)
```

Subcommands: - /subagents (alias of list) — table: ID STATE ISOLATION DURATION DESCRIPTION (description, NEVER prompt body — CONST-042). - /subagents status <id> — full record minus prompt body. - /subagents kill <id> — calls manager.Kill(id).

Tests:

```
func TestSubagentsCommand_List_RendersTable(t *testing.T) { /*
    fake mgr returns 2 infos; output contains both IDs and
    STATE column */ }

func TestSubagentsCommand_Status_ShowsDescriptionNotPromptBody(t
    *testing.T) {
    // CONST-042 anti-leak.
    fake := &fakeSAMgr{statusInfo: subagent.SubagentInfo{ID:
        "x", Description: "do thing"}}
    cmd := NewSubagentsCommand(fake)
    res, _ := cmd.Execute(context.Background(),
        &CommandContext{Args: []string{"status", "x"}})
    require.Contains(t, res.Output, "do thing")
    require.NotContains(t, res.Output, "SECRET-PROMPT-BODY",
        "prompt body MUST NEVER appear in slash output")
}
```

```

func TestSubagentsCommand_Kill_DelegatesToManager(t *testing.T)
    { /* fake.killCalled[id] == true */ }
func TestSubagentsCommand_Kill_NoArgs_ReturnsUsageError(t
    *testing.T) {}
func TestSubagentsCommand_NilManager_ReportsUnavailable(t
    *testing.T) { /* ".../subagents subagent manager
    unavailable" */ }

```

Subject: feat(P1-F15-T09): /subagents slash command (list/status/kill) + CONST-042 anti-leak.

Task 10: main.go wiring + integration test

Files: modify helix_code/cmd/cli/main.go, modify helix_code/internal/tools/registry.go, new helix_code/tests/integration/subagent_test.go (//go:build integration).

registry.go modifications:

```

const CategorySubagent ToolCategory = "subagent"

subagentManager *subagent.SubagentManager // optional; nil
    disables `task` registration

func (r *ToolRegistry) SetSubagentManager(m
    *subagent.SubagentManager) {
    r.mu.Lock(); defer r.mu.Unlock()
    r.subagentManager = m
    r.tools["task"] = NewTaskTool(m) // direct registration; no
        adapter needed
}

```

main.go startup wiring per spec §4.1.

Integration tests (gated):

```

//go:build integration
// +build integration

func TestSubagent_InProcess_RealLLM_RealToolRegistry(t
    *testing.T) {
    // FakeLLMProvider + real ToolRegistry
}

func TestSubagent_Subprocess_RealForkExec_FakeLLM(t *testing.T) {
    // Runs the host binary with HELIX_SUBAGENT_INVOCATION=1 +
        payload
    // Asserts child PID != os.Getpid(); stdout decodes;
        result.State=succeeded
}

```

```

}

func TestSubagent_Worktree_RealGitWorktree(t *testing.T) {
    if _, err := exec.LookPath("git"); err != nil {
        t.Skip("SKIP-OK: P1-F15 git not on PATH (install: apt
            install git OR dnf install git)")
    }
    // git init tempdir, dispatch worktree-isolated subagent
    // assert result.WorktreePath exists; rev-parse --show-
        toplevel inside == result.WorktreePath
}

func TestSubagent_StreamingOrder(t *testing.T) {
    // FakeLLMProvider with per-prompt sleep; A fast B slow → A's
        result first.
}

func TestSubagent_Kill_Subprocess(t *testing.T) {
    if _, err := exec.LookPath("git"); err != nil {
        t.Skip("SKIP-OK: P1-F15 git not on PATH (install: apt
            install git OR dnf install git)")
    }
    // dispatch slow subagent, Kill(id), assert
        result.State=StateKilled within 6s.
}

```

Subject: feat(P1-F15-T10): wire SubagentManager into main.go + /
subagents + gated integration tests.

Task 11: Challenge with runtime evidence

Files: new helix_code/tests/integration/cmd/p1f15_challenge/main.go,
new challenges/p1-f15-subagent-team/CHALLENGE.md, new challenges/p1-
f15-subagent-team/run.sh.

Output skeleton:

```

=== IN-PROCESS + FakeLLM (always runs) ===
[PASS] in-process: dispatch returns id+chan
[PASS] in-process: FakeLLMProvider.GenerateCallCount == 1 (anti-bluff: pro
[PASS] in-process: result.Output == canned response (anti-bluff: NOT promp
[PASS] in-process: channel closes after WaitAll drains
[PASS] in-process: duration > 0

=== SUBPROCESS + FakeLLM (always runs) ===
[PASS] subprocess: child PID != os.Getpid() (anti-bluff: real fork-exec)
[PASS] subprocess: HELIX_SUBAGENT_INVOCATION=1 sentinel observed in child
[PASS] subprocess: stdout JSON decodes to SubagentResult
[PASS] subprocess: result.State == succeeded

```

```
=== WORKTREE + Subprocess + FakeLLM (gated) ===
[PASS|skipped: git not on PATH (install: apt install git OR dnf install git)]
[PASS] worktree: result.WorktreePath exists (os.Stat succeeds)
[PASS] worktree: result.WorktreePath != parent cwd (anti-bluff: real isolation)
[PASS] worktree: git rev-parse --show-toplevel inside path == path
[PASS] worktree: subagent ran with cmd.Dir set to worktree path
```

```
=== REAL LLM (gated) ===
[PASS|skipped: ANTHROPIC_API_KEY unset (export ANTHROPIC_API_KEY=...)]
[PASS] real-llm: provider returned non-empty text
[PASS] real-llm: text contains literal "p1f15-real-llm-ok"
```

SUMMARY: IN-PROCESS=5/5 PASS; SUBPROCESS=4/4 PASS; WORKTREE=<n>/4 PASS; REAL-LLM=1/1 PASS

The Challenge MUST exit non-zero on any assertion failure within phases that did run. Anti-bluff smoke clean check appended to harness output. Verbatim output captured into 06_phase_1_evidence.md. Dual commit (Challenges submodule + meta-repo bump).

Subject: feat(P1-F15-T11): challenge with runtime evidence (in-process + subprocess always; worktree + real-LLM gated).

Task 12: Close-out + push

Tick all 12 items in PROGRESS, advance PROGRESS focus to F16 candidate, run final verification (make verify-compile, anti-bluff smoke, go test -count=1 ./internal/agent/subagent/... ./internal/tools/... ./internal/commands/...), commit chore(P1-F15-T12): close out feature 15 – subagent team, push 4 remotes non-force (origin, helixdev, vasic-digital, gitlab per programme conventions).

Self-review notes

1. **Spec coverage:** every spec section maps to a task — T02 types + FakeLLMProvider (§3.3, §5.2), T03 in-process spawner (§4.2, §5.2 criterion 1), T04 subprocess spawner (§4.3, §5.2 criterion 2), T05 manager + streaming (§4.4, §5.2 criterion 4), T06 worktree integration (§4.5, §5.2 criterion 3), T07 task tool (§3.3, §3.4), T08 helper-mode + main.go (§4.1), T09 / subagents slash (§3.4, §9 CONST-042), T10 wiring + integration (§4.1, §6.2), T11 Challenge four phases (§5.2, §6.3), T12 close-out (§9).
2. **TDD:** every code task starts with a failing test that exercises real code paths (FakeLLM call-count assertion; subprocess argv+env recording; worktree dir existence + git rev-parse --show-toplevel; channel-close-on-error invariant).
3. **Type consistency:** SubagentTask, SubagentResult, Isolation, SubagentState, SubagentSpawner, SubagentManager, Subagent, SubagentInfo, InProcessSpawner, SubprocessSpawner, TaskTool, SubagentsCommand, FakeLLMProvider, env-var names — all match across spec §3.3 and plan T02–T09.
4. **No new external deps:** os/exec, os.Executable, encoding/json, os.Getenv are stdlib; github.com/google/uuid v1.6.0 and go.uber.org/zap v1.27.0 already in go.mod. No go get needed.
5. **Anti-bluff (§5.2):** Challenge has FOUR phases; A and B always run; C is gated on git; D is gated on ANTHROPIC_API_KEY. The four real-execution criteria each have a

- dedicated PASS line. The FakeLLMProvider's anti-misuse comment is itself self-tested in T02. The bluff scanner gets a special-case allowlist for FakeLLMProvider in `internal/agent/subagent/types.go`; ANY other "fake provider" outside `_test.go` files in production paths is a bluff.
6. **CONST-042:** `SubagentTask.Prompt` logged at DEBUG only; INFO logs use `task.Description`. `/subagents status` prints description, never prompt body — asserted by `TestSubagentsCommand_Status_ShowsDescriptionNotPromptBody`.
 7. **CONST-043:** stays on `main`, non-force to all four remotes; explicit user authorization is requested at T12 before pushing.
 8. **F04 worktree API constraint** (non-obvious): `worktree.Manager.EnterWorktree` uses `git worktree add <path> <branch>` which leaves the parent's WIP in the parent — uncommitted changes are NOT copied into the new worktree. Subagents start from the clean tip of `BaseBranch` (or `HEAD` if empty). v1 documents this loudly in spec §4.5; auto-stash-on-dispatch is deferred to F15.5. Additionally, `EnterWorktree` mutates the manager's `currentWorktree` state (it's used by F04 to track the agent's own location) — F15 must NOT call `EnterWorktree` on a shared manager that the parent agent is also using to track its own location, because the call will move the parent's tracked location into the subagent's worktree. Mitigation: `prepareWorktreeForSubagent` calls a new `wtMgr.CreateWorktreeForSubagent(ctx, name, baseBranch)` helper added in T06 that does the git work without mutating `currentWorktree`. If F04's API does not expose such a helper, T06 adds it as a 5-line method on `worktree.Manager` (forking the existing `EnterWorktree` body but skipping the `m.currentWorktree = path` assignment) and updates F04's tests accordingly.
 9. **Subagent recursion guard (v1):** the subprocess child's `RunAsSubagent` does NOT register the `task` tool in its inner `ToolRegistry`. This caps subagent depth at 1 in v1. F15.5 may lift with a depth-limit ENV var.
 10. **Branch + push:** stays on `main`, non-force to all four remotes (per CONST-043); explicit user authorization is requested at T12 before pushing.
 11. **Reality check:** the existing `Tool` interface in `internal/tools/registry.go` (`Name/Description/Schema/Execute/Category/Validate`) is fully compatible with `TaskTool`. The `SetSubagentManager` shape matches the existing F13 `SetLSPManager` and F14's `SetSandboxManager`, so the registry change is one method + one constant + one lazy-registration line. No registry redesign.
 12. **Helper-mode dispatch ordering:** `subagent.IsSubagentInvocation()` MUST be the FIRST statement of `main()` — before `sandbox.IsHelperInvocation()` — because a subagent subprocess might itself spawn a sandboxed shell, but the inverse (a sandbox helper spawning a subagent) is not a thing in v1. The ordering is documented in `main.go` as an anti-bluff anchor comment.